Assignment 2: Convolutional Neural Networks

Jiajun Shen | js6890 | APAN PS5560 | October 12, 2026

1. CNN architecture

The model follows the specified architecture. PyTorch stores images as channels first, so one RGB image has shape 3 x 64 x 64. Shapes below omit the batch dimension.

Layer	Output shape	Parameters
Input	3 x 64 x 64	0
Conv2d(3, 16, 3, stride=1, padding=1)	16 x 64 x 64	448
ReLU; MaxPool2d(2, stride=2)	16 x 32 x 32	0
Conv2d(16, 32, 3, stride=1, padding=1)	32 x 32 x 32	4,640
ReLU; MaxPool2d(2, stride=2)	32 x 16 x 16	0
Flatten	8,192	0
Linear(8192, 100); ReLU	100	819,300
Linear(100, 10)	10	1,010
Total		825,398

The last layer returns logits for CrossEntropyLoss. Softmax is applied only when the API returns class probabilities. The implementation is in app/cnn.py.

2. CIFAR-10 training

The 50,000 official training images were split into 45,000 training and 5,000 validation images using seed 42. The official 10,000 test images were held out until the best validation checkpoint was selected. All images were resized from 32 x 32 to 64 x 64 and normalized with mean and standard deviation 0.5 in each channel. Training also used random cropping with padding 4 and horizontal flipping.

Training used AdamW, an initial learning rate of 0.001, weight decay 0.0001, batch size 256, and a cosine learning-rate schedule for 15 epochs on Apple MPS. The architecture was not changed.

Measure	Result
Selected epoch	15
Validation accuracy	64.52%
Test accuracy	65.18% (6,518 / 10,000)
Test cross-entropy loss	0.9886

The checkpoint, per-epoch results, and test metrics are saved in models/. The model is a small CIFAR-10 classifier; its softmax scores are not calibrated confidence estimates for arbitrary photos.

3. API and Docker deployment

The new POST /classify endpoint accepts an uploaded image and returns a class ID, class name, softmax score, and all ten class probabilities. The existing /, /generate, and /embedding endpoints are preserved. The saved checkpoint is included in the Docker image and loaded once at startup; prediction runs on CPU in evaluation mode with gradient tracking disabled.

Code: github.com/xiangbuchu77/sps-genai-assignment1
Branch: codex/assignment2-cnn

From a terminal:

```
git clone --branch codex/assignment2-cnn --single-branch \
  https://github.com/xiangbuchu77/sps-genai-assignment1.git
cd sps-genai-assignment1
docker build -t sps-genai-assignment2 .
docker run --rm -p 127.0.0.1:8000:80 sps-genai-assignment2
```

Open http://127.0.0.1:8000/docs, expand POST /classify, select Try it out, upload an image, and select Execute. The first build requires Internet access to install dependencies, including the existing spaCy model. A GPU and retraining are not required to serve predictions.

A sample request from a second terminal:

```
curl -X POST http://127.0.0.1:8000/classify \
  -F "file=@examples/cifar10_test_0000.png"
```

Observed response from the Docker container (probabilities abbreviated):

```
{
  "class_id": 3,
  "class_name": "cat",
  "confidence": 0.5339770913124084
}
```

The example is official CIFAR-10 test image 0, whose true class is cat. The predicted class was cat. The example index was fixed before checking its prediction.

The Docker container started successfully and the health check passed. HTTP checks covered the new classifier, the existing text and embedding endpoints, and invalid image uploads. The endpoint rejects empty or unreadable files and files larger than 5 MiB, and converts grayscale/RGBA images to RGB.

Reproduce training

```
uv sync --frozen
uv run python train_cnn.py --epochs 15 --batch-size 256 --seed 42
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000
```

Training downloads CIFAR-10 into data/. It saves the checkpoint with the highest validation accuracy and evaluates it once on the test set. The default device is CUDA, MPS, or CPU according to availability. Exact results can vary across devices.

4. Theory: arithmetic of CNNs

For a convolution with dilation 1, the output size along one spatial dimension is:
O = floor((N + 2P - K) / S) + 1
Here N is the input size, P is padding on each side, K is kernel size, and S is stride. The same formula applies to ordinary max pooling with its default floor behavior.

Question 1

Input: 32 x 32 x 3; eight 5 x 5 filters; stride 1; no padding.
O = floor((32 + 0 - 5) / 1) + 1 = 28.
Each filter produces one output channel, so the output is **28 x 28 x 8**.

Question 2

With same padding and stride 1, P = (5 - 1) / 2 = 2 on each side.
O = floor((32 + 2(2) - 5) / 1) + 1 = 32.
The output becomes **32 x 32 x 8**; the number of filters is unchanged.

Question 3

Input: 64 x 64; a 3 x 3 filter; stride 2; no padding.
O = floor((64 - 3) / 2) + 1 = floor(30.5) + 1 = 31.
The output spatial size is **31 x 31**. The last incomplete window is not used.

Question 4

Input: 16 x 16; max-pooling kernel 2 x 2; stride 2; no padding.
O = floor((16 - 2) / 2) + 1 = 8.
The output is **8 x 8** for each feature map. Pooling preserves the channel count.

Question 5

A 3 x 3 convolution with same padding and stride 1 uses P = 1.
First layer: floor((128 + 2 - 3) / 1) + 1 = 128.
Second layer: floor((128 + 2 - 3) / 1) + 1 = 128.
The final spatial shape is **128 x 128**. The output channel count equals the second layer's filter count, which the question does not specify.

Question 6

model.train() sets the model and its submodules to training mode. It does not perform optimization or enable gradients by itself. A newly created PyTorch model starts in training mode, so removing the call may have no effect if model.eval() has never been called. After validation, however, removing it leaves the model in evaluation mode: Dropout stays disabled and BatchNorm uses its stored statistics instead of updating them. Backpropagation and optimizer steps can still run.

This assignment's CNN contains only convolution, ReLU, pooling, flattening, and linear layers. These layers behave the same in training and evaluation modes, so omitting model.train() alone does not change this model's forward computation. The explicit call is still useful when resuming training after evaluation.

References: PyTorch nn.Module (train/eval); Torchvision CIFAR10.